

Módulo 6 — Ambiente, Higiene y Servicios Urbanos

Módulo operativo de campo de la higiene urbana y el control ambiental. Planifica y ejecuta los servicios de recolección, limpieza, arbolado y espacios verdes; administra el inventario urbano ambiental; y tramita las denuncias ambientales hasta el acta de constatación, registrando después la resolución sancionatoria que devuelve M4.

Índice

1. Qué hace el módulo	
2. Glosario	
3. Funcionalidades	
Configuración de catálogos	
Programación de servicios	
Ejecución en campo	
Residuos	
Limpieza viaria	
Contenedores y puntos verdes	
Arbolado	
Espacios verdes	
Control ambiental	
Seguimiento del ciudadano	
Tablero de indicadores	
4. Entidades y estados	
4.1 El concepto central: `Service`	
4.2 Entidades	
4.3 Estados y transiciones	
5. Eventos que publicamos	
5.1 Qué información lleva cada evento	
5.2 Los siete que no publicamos	
5.3 Enumeraciones	
6. Eventos y datos que necesitamos recibir	
6.1 Campos imprescindibles	
7. Puntos bloqueantes	

1. Qué hace el módulo

Área	Responsabilidad
Configuración de catálogos	Tipos de servicio, zonas operativas, frecuencias y recorridos
Programación de servicios	Agenda sobre recorridos u objetivos puntuales; reprogramación y cancelación
Ejecución en campo	Inicio, avance, suspensión y cierre; registro de lo que quedó sin atender y por qué
Residuos	Recolección domiciliaria, de reciclables, de voluminosos y de verdes; volumen, peso y destino final
Limpieza viaria	Barrido manual y mecánico, y lavado de calles, como servicio programable sobre recorrido
Contenedores y puntos verdes	Alta, ubicación, capacidad, desborde, daño, vaciado, reparación y reubicación. Puntos verdes de entrega voluntaria
Arbolado	Censo con historial de relevamientos; poda, extracción, plantación y tratamiento
Espacios verdes	Plazas y parques, con riego y corte de césped
Control ambiental	Denuncias ambientales —ruidos, vertidos, microbasurales, emisiones—, inspección, acta de constatación y violación constatada
Seguimiento del ciudadano	Publicamos el avance de la inspección para que M2 lo muestre, y ofrecemos una vista de consulta complementaria con el detalle operativo

2. Glosario

Término	Qué es en M6
Zona operativa (<code>Zone</code>)	Agrupación propia que junta uno o más barrios para armar recorridos y asignar cuadrillas
Servicio (<code>Service</code>)	Unidad de trabajo programable: tipo, fecha, cuadrilla y estado. Puede ser <code>ROUTE</code> o <code>POINT</code>
Denuncia	El reclamo que el vecino presenta en M2, dueño del registro. En el vocabulario de M2 es un ticket , y ese es el nombre que usamos en eventos y campos (<code>ticketId</code>)
Acta (<code>ViolationNotice</code>)	Lo que emitimos al constatar una violación. Inmutable una vez emitida
Cuadrilla (<code>Crew</code>)	Equipo de trabajo, municipal o de cooperativa, que ejecuta servicios

3. Funcionalidades

Configuración de catálogos

- Alta, edición y baja de tipos de servicio, zonas operativas y recorridos.
- Definición de frecuencias, cuadrillas, vehículos y sitios de disposición final.

Programación de servicios

- Generación automática de la agenda a partir de recorrido y frecuencia, para un rango de fechas.
- Servicios puntuales sobre un bien del inventario o una ubicación, manuales o derivados de un reclamo.
- Asignación de zona a toda programación: se copia del recorrido, se toma del bien, o se deduce del barrio cuando el objetivo es una dirección suelta.
- Asignación de cuadrilla y vehículo, con validación de disponibilidad y de no superposición.
- Reprogramación individual y masiva —esta última ante alerta meteorológica— y cancelación, ambas con motivo obligatorio.
- Vista de agenda por cuadrilla, zona y tipo de servicio.

Ejecución en campo

- Inicio, avance, suspensión, reanudación y cierre del servicio.
- Resultado por zona en los recorridos, o resultado único en los servicios puntuales.
- Registro de lo no atendido, con motivo obligatorio y propuesta de nueva fecha.
- Adjuntos y notas a nivel de servicio y de zona.
- Marcado automático de servicios demorados según una tolerancia configurable.

Residuos

- Gestión diferenciada de residuos domiciliarios, reciclables, voluminosos y verdes.
- Registro de volumen, peso y sitio de disposición al cerrar el servicio.
- Retiros puntuales de voluminosos originados en un reclamo.
- Consulta histórica por zona, tipo de residuo, destino y período.

Limpieza viaria

- Barrido manual, barrido mecánico y lavado de calles como servicio recurrente sobre recorrido.
- Mismo ciclo de ejecución y mismo resultado por zona que el resto de los servicios: no es un flujo aparte.

Contenedores y puntos verdes

- Alta, edición y baja de contenedores, con código, tipo, ubicación, capacidad, zona y estado.
- Registro de desborde y de daño, con tipo de daño y severidad.
- Programación de vaciado, limpieza, reparación y reubicación.
- Puntos verdes de entrega voluntaria, con materiales aceptados y estado, y retiro programado de lo acopiado hacia planta de reciclaje.
- Listado por zona, con estado actual e historial.

Arbolado

- Censo de árboles con especie, ubicación, altura y diámetro.
- Relevamientos con estado sanitario y nivel de riesgo, conservando el historial completo.
- Generación automática de intervención urgente cuando el riesgo es alto o crítico.
- Poda de formación, poda de seguridad, extracción, plantación y tratamiento.
- La extracción exige autorización previa de un supervisor, con justificación.
- Indicación de necesidad de corte de calle y generación de la solicitud a M7.

Espacios verdes

- Alta, edición y baja de plazas, parques, canteros y espacios similares, con superficie, zona y estado.
- Riego y corte de césped como servicio recurrente.
- Registro de incidencias y derivación a M3.

Control ambiental

- Registro del expediente técnico a partir de una denuncia derivada de M2 o de una detección de oficio.
- Priorización, y elevación de prioridad ante escalamiento del reclamo.
- Inspecciones con checklist, hallazgos, adjuntos y resultado.
- Emisión del acta de constatación, con numeración correlativa e inmutable una vez emitida. Solo se deriva a M4 si identificamos el establecimiento.
- Registro de la resolución sancionatoria que devuelve M4 —multa, clausura o desestimación—, que es lo que cierra el expediente.
- Desestimación con motivo, y cierre con resolución.
- Marcado de un expediente como basural recurrente, con propuesta de limpieza programada.

Seguimiento del ciudadano

- Publicación del avance de la inspección hacia M2, que es el canal oficial del vecino. Nuestra vista es complementaria: se alimenta de los mismos eventos, así que no pueden divergir.
- Vista de consulta: listado y detalle de las denuncias ambientales que el ciudadano presentó en M2, con estado actual, inspección asociada y fecha estimada de atención.
- La fecha estimada es la fecha programada del servicio o de la inspección. **No calculamos SLA:** el plazo comprometido con el vecino lo fija M2.
- Proyección pública restringida, tanto en la vista como en lo que publicamos por evento: nunca notas internas, contenido del acta, identidad del inspector ni datos de terceros.
- Las denuncias anónimas no son consultables por este canal. El ciudadano no crea, modifica ni cierra nada en este módulo.

Tablero de indicadores

- **Cobertura:** objetivos atendidos sobre programados, por período, zona y tipo de servicio.
- **Cumplimiento:** servicios finalizados en término, demorados, y ranking de zonas no atendidas con sus motivos.
- **Incidencias:** contenedores desbordados y dañados por zona, árboles por nivel de riesgo, denuncias por tipo y estado, y tiempo medio de resolución.
- **Residuos:** toneladas y metros cúbicos por tipo y destino, y porcentaje desviado a reciclaje.

4. Entidades y estados

4.1 El concepto central: `Service`

Un `Service` es toda unidad de trabajo programable del módulo: siempre tiene tipo, fecha, ventana horaria, cuadrilla, vehículo si corresponde, estado y evidencia. Lo que varía es sobre qué se ejecuta.

mode	Objetivo	Ejemplo	Resultado
<code>ROUTE</code>	Secuencia ordenada de zonas	Recolección domiciliaria Zona Norte	Un <code>ZoneResult</code> por zona
<code>POINT</code>	Un bien del inventario o una ubicación	Vaciar el contenedor CT-0442; podar el árbol TR-01293	Un único resultado

Todo `Service` lleva `zoneIds[]` y nunca viene vacío: las zonas del recorrido en `ROUTE`, una sola en `POINT`. Se copia al programar y no se recalcula, para que editar un recorrido no altere lo ya ejecutado.

En `POINT` la zona sale del bien del inventario, o del barrio de la dirección si el objetivo es una ubicación suelta.

Las podas y las inspecciones ambientales también son servicios: `TreeIntervention` y `EnvironmentalInspection` guardan qué hay que hacer, y el `Service` cuándo, con qué cuadrilla y cómo terminó.

4.2 Entidades

Configuración y planificación

Entidad	Campos principales
<code>ServiceType</code>	<code>code</code> , <code>name</code> , <code>category</code> , <code>mode</code> , <code>requiresVehicle</code>
<code>Zone</code>	Zona operativa propia. <code>code</code> , <code>name</code> , <code>neighborhoodIds[]</code> , <code>status</code>
<code>Route</code>	<code>code</code> , <code>name</code> , y la secuencia de <code>RouteStop</code> (<code>sequence</code> , <code>zoneId</code> , <code>estimatedDurationMin</code>)
<code>ServiceFrequency</code>	<code>serviceTypeId</code> , <code>routeId</code> , <code>weekdays[]</code> , <code>shift</code> , <code>validFrom</code> , <code>validTo</code>

Recursos

Entidad	Campos principales
<code>Crew</code>	<code>name</code> , <code>crewType</code> , <code>leaderUserId</code> , <code>memberUserIds[]</code> , <code>organizationId</code> , <code>defaultShift</code> , <code>status</code>
<code>Vehicle</code>	<code>plate</code> , <code>vehicleType</code> , <code>capacity</code> , <code>status</code>

Ejecución

Entidad	Campos principales
<code>Service</code>	<code>serviceTypeId</code> , <code>mode</code> , <code>zoneIds[]</code> , <code>routeId</code> <code>targetRef</code> , <code>scheduledDate</code> , <code>timeWindow</code> , <code>crewId</code> , <code>vehicleId</code> , <code>status</code> , <code>statusReason</code> , <code>origin</code> , <code>ticketId</code> , <code>attachments[]</code> , <code>notes</code> . <code>crewId</code> es opcional hasta que se asigna la cuadrilla
<code>ZoneResult</code>	Hijo de un <code>Service</code> con <code>mode = ROUTE</code> , uno por zona. <code>zoneId</code> , <code>status</code> , <code>reason</code> , <code>proposedDate</code> , <code>notes</code> , <code>attachments[]</code> , <code>recordedAt</code>

Entidad	Campos principales
CollectionRecord	wasteType , volumeM3 , weightKg , disposalSiteId
DisposalSite	code , siteType , name

Inventario urbano

Entidad	Campos principales
Container	code , containerType , location , capacityLiters , zoneId , status , y con el estado DAMAGED también damageType , severity , requiresPublicWorks
GreenPoint	Punto verde de entrega voluntaria. code , name , location , acceptedWasteTypes[] , zoneId , status
Tree	surveyCode , species , location , heightM , diameterCm , zoneId , más el historial de TreeSurvey
TreeSurvey	treeId , surveyedAt , inspectorId , healthStatus , riskLevel , riskType , suggestedIntervention , requiresStreetClosure , requiresPublicWorks , notes
TreeIntervention	interventionType , treeIds[] , location , requiresStreetClosure , status , priority , serviceId , y en las extracciones authorizedByUserId , authorizedAt , justification
GreenSpace	name , spaceType , areaM2 , zoneId , status

Control ambiental

Entidad	Campos principales
EnvironmentalReport	reportType , location , ticketId , reporterSnapshot , status , priority
EnvironmentalInspection	reportId , serviceId , inspectorId , inspectedAt , checklist[] , findings , attachments[] , outcome
ViolationNotice	El acta. noticeNumber , inspectionId , issuedAt , establishmentId , violationType , severity , suggestedAction . Immutable
SanctionOutcome	La resolución de M4, espejada de solo lectura para poder cerrar el expediente. violationNoticeId , decision , decidedAt , externalRef , dismissalReason . M4 confirmó (24/08) que decidedAt y externalRef vuelven a su payload

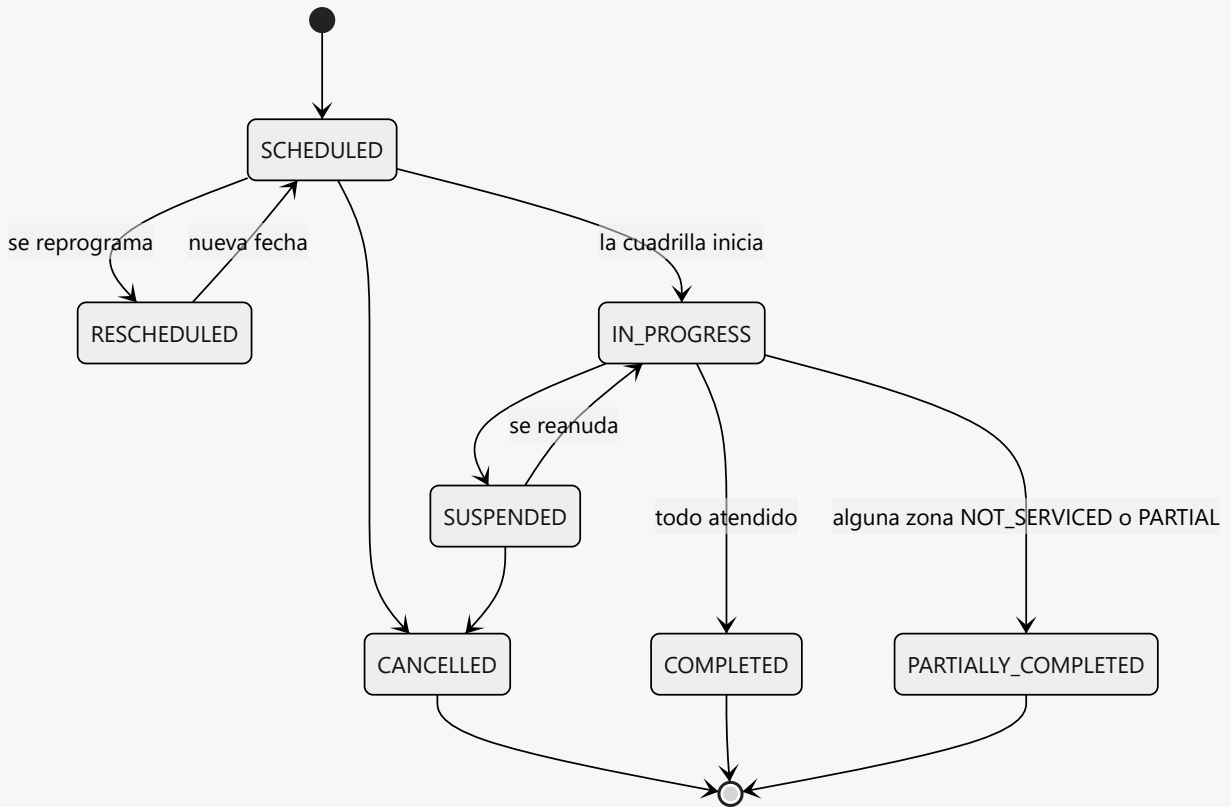
Derivaciones salientes

Entidad	Campos principales
RepairRequest	Hacia M3. damageType , location , severity , detectedIn , status
StreetClosureRequest	Hacia M7. reason , sourceRef , streets[] , from , to , status

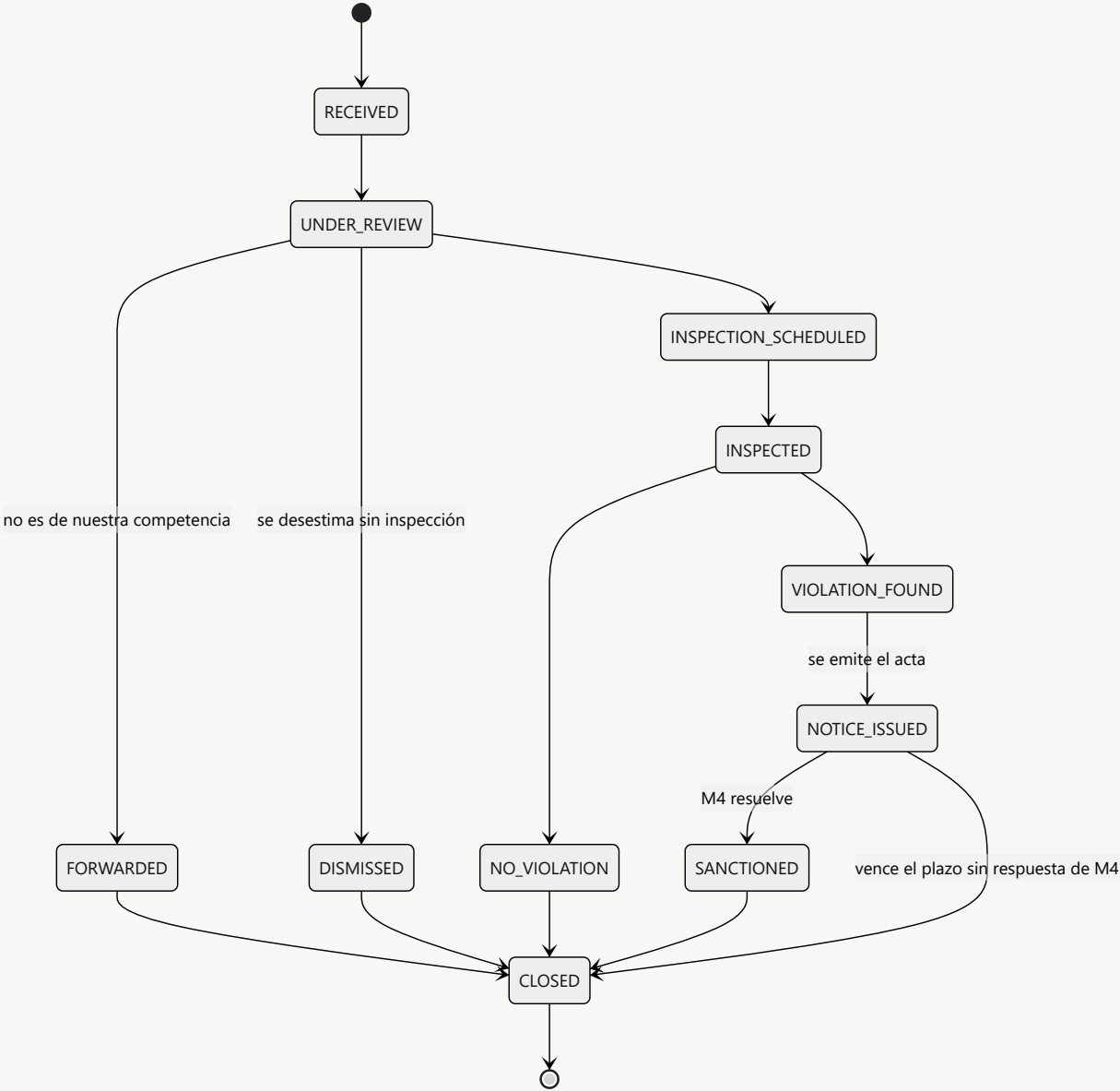
4.3 Estados y transiciones

Estos son los estados que un módulo consumidor puede llegar a ver reflejados en nuestros eventos.

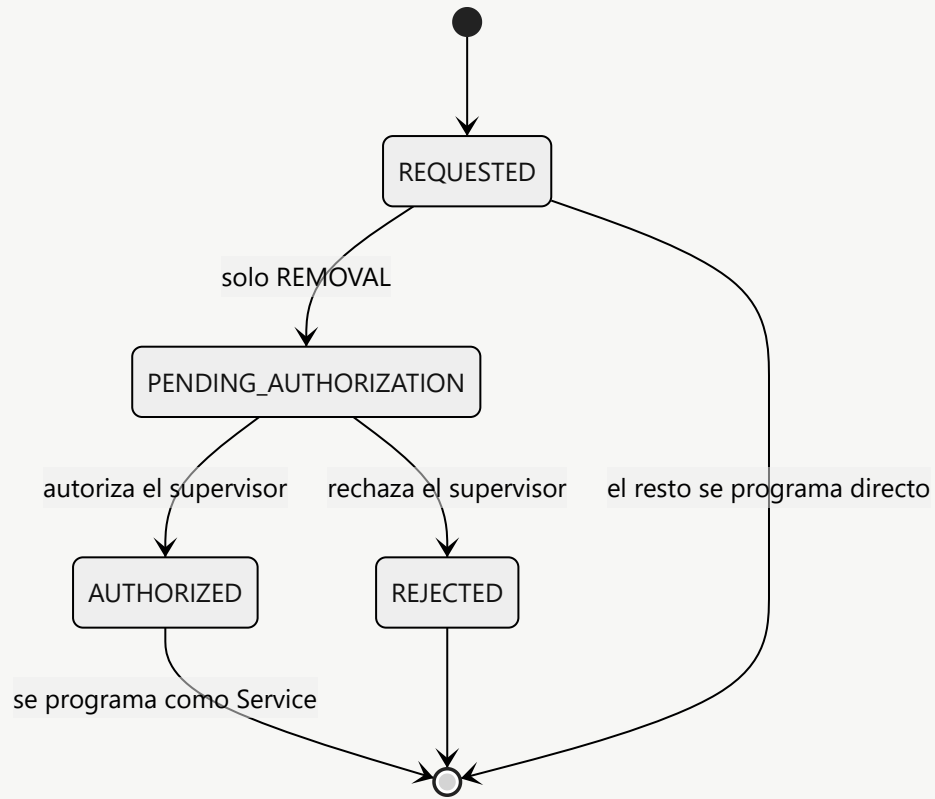
Service — el mismo diagrama vale para `mode = ROUTE` y para `mode = POINT`



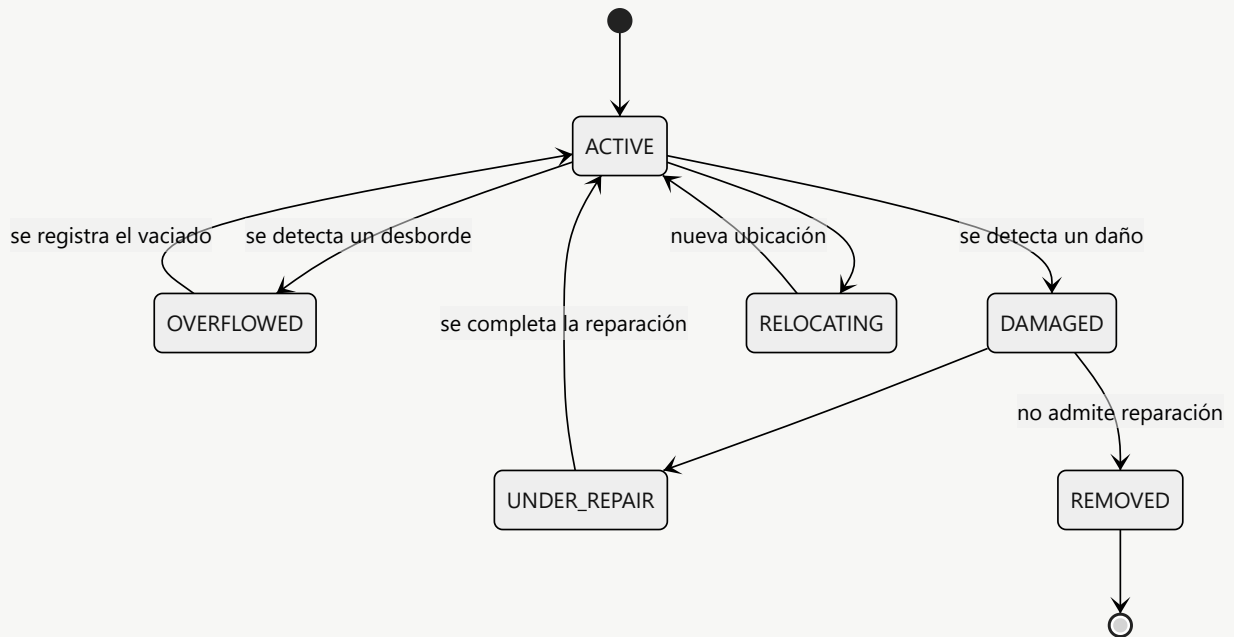
DELAYED no es un estado, es un aviso puntual: `urbanServiceDelayed` se dispara una vez y el servicio sigue en **SCHEDULED** o **IN_PROGRESS**.



TreeIntervention — la extracción necesita autorización; el resto no



Container



5. Eventos que publicamos

Ocho eventos, y los ocho tienen consumidor confirmado.

Evento	Se dispara cuando	Consumidores
<code>updateTicketStatus</code>	Cambia el estado de un servicio o una inspección nacidos de un reclamo	M2
<code>urbanServiceScheduled</code>	Se agenda un servicio con fecha, zona y cuadrilla	M7
<code>containerDamaged</code>	Se detecta un contenedor dañado o faltante	M3
<code>treeRiskDetected</code>	Un relevamiento arroja riesgo <code>HIGH</code> o <code>CRITICAL</code>	M3, M7
<code>treePruningScheduled</code>	Se programa una intervención de poda	M7
<code>environmentalViolationDetected</code>	Se emite un acta ambiental	M4
<code>infrastructureRepairRequested</code>	Detectamos un daño de infraestructura ajeno a nuestra competencia	M3
<code>streetClosureRequested</code>	Un servicio o intervención requiere cortar la calle	M7

Eventos que decidimos no publicar

Evento	Por qué no
<code>urbanServiceStarted</code> , <code>urbanServiceDelayed</code> , <code>urbanServiceCompleted</code> , <code>zoneNotServiced</code> , <code>containerOverflowed</code> , <code>environmentalInspectionScheduled</code> , <code>environmentalInspectionCompleted</code>	Estaban en el contrato porque se los ofrecíamos a M2 como detalle del avance de un reclamo. Cuando M2 definió que todo lo suyo entra por <code>updateTicketStatus</code> , se quedaron sin ningún consumidor. Los hechos siguen existiendo en nuestro modelo —son los que disparan cada <code>updateTicketStatus</code> — pero no salen al bus: un evento que nadie escucha es schema, publisher, test y documentación para nadie. El payload de cada uno está diseñado: si alguno hace falta, se pide y se publica
<code>environmentalFineIssued</code>	No generamos cargos económicos. Nuestra acta llega a M5 a través de M4, convertida en <code>commercialFineGenerated</code>
<code>environmentalReportCreated</code>	El dueño de la denuncia es M2; que abramos un expediente interno no le agrega nada. Lo que sí necesita es la inspección programada y su resultado, que publicamos
<code>citizenNotificationRequested</code>	Las notificaciones las resuelve M9, aunque todavía falta definir qué las dispara: varios módulos consumimos <code>notificationSent</code> y ninguno publica algo que la origine

5.1 Qué información lleva cada evento

Información de negocio, no formato: el sobre lo define M9. Diseño tentativo — `?` es opcional o puede venir en nulo, `[]` es lista. Los valores de cada enumeración están en 5.3.

`urbanServiceScheduled`

```
serviceId, serviceTypeCode, category, mode,  
zoneIds[], routeId?, targetRef?,  
scheduledDate, timeWindow { from, to },  
crewId?, vehicleId?,  
origin, ticketId?
```

`zoneIds[]` : una sola zona en `POINT` , las del recorrido en `ROUTE` . `targetRef` : el bien del inventario, solo en `POINT` .
`ticketId` : viaja siempre que `origin = TICKET` .

containerDamaged

```
containerId, containerCode, zoneId, location,  
damageType, severity, requiresPublicWorks,  
detectedAt, ticketId?
```

`requiresPublicWorks = true` marca el componente de infraestructura civil ajeno a nosotros: es la señal para M3.

treeRiskDetected

```
treeId, surveyCode, species, zoneId, location,  
riskLevel, riskType, healthStatus,  
suggestedIntervention,  
requiresStreetClosure, requiresPublicWorks,  
surveyedAt
```

Solo se dispara con `riskLevel` en `HIGH` o `CRITICAL` . M3 lo aceptó en su lista de consumo, así que queda descartado el evento genérico con `hazardType` que ofrecíamos como alternativa. Su lista actualizada ya no tiene `urbanRiskDetected` .

treePruningScheduled

```
interventionId, serviceId, interventionType, treeIds[],  
zoneId, location,  
scheduledDate, timeWindow { from, to },  
crewId, requiresStreetClosure
```

La fecha, la ventana y la cuadrilla salen del `Service` , que va identificado por si quieren seguirlo con los eventos de servicio.

environmentalViolationDetected → **M4**

```
violationId, noticeNumber, issuedAt,  
reportId, inspectionId, ticketId?,  
violationType, severity, location,  
establishmentId,  
priorNoticeCount,  
evidence[], suggestedAction
```

`establishmentId` es obligatorio: es lo único sobre lo que M4 puede actuar, porque intimar, clausurar y multar se le aplican a un comercio habilitado. Si no identificamos el establecimiento no derivamos el acta y el expediente cierra de nuestro lado.
`suggestedAction` **no es vinculante**.

infrastructureRepairRequested → **M3**

```
requestId, damageType, severity, location,
detectedIn, ticketId?,
publicSafetyRisk, requestedAt
```

✓ Pedimos que M3 devuelva `requestId` como `sourceRequestId` al cerrar la reparación, y ya lo confirmaron —en `workOrderScheduled` y en `workOrderCompleted`.

streetClosureRequested → **M7**

```
requestId, sourceModule, sourceRef,
reason, streets[], from, to,
closureType, requestedAt
```

`sourceRef` apunta al servicio o a la intervención que origina el corte. ✓ **25/08:** M7 confirmó que devuelven el origen — `requestId` viaja como `closureRequestId` y `sourceModule` como `requestingModule` ("Ambiente", en nuestro caso) — en `streetClosureApproved` y `streetClosureRejected`. `streetClosureEnded` es la excepción y no los trae.

updateTicketStatus → **M2**

```
ticketId, updateType,
publicMessage?, internalMessage?, progress?, details?,
attachments[]?, updatedBy, statusChangedAt
```

Es el canal del vecino, y el payload lo define M2 en su contrato, ahora en su **v1.5** (reemplazó la v1.2 sobre la que se había escrito esto): lo adoptamos tal cual. Lo publicamos **solo cuando el servicio o la inspección nacieron de un reclamo**: un servicio planificado no genera nada hacia M2. No lleva estado —informamos el hecho con `updateType` y M2 decide la transición— ni `sourceRef`, porque su contrato prohíbe transportar IDs de nuestras entidades internas: la correlación `ticketId ↔ serviceId` queda de nuestro lado. 📦 La v1.5 sacó `publicId / expectedTicketVersion` del contrato, partió `message` en `publicMessage / internalMessage` y renombró `updatedAt` a `statusChangedAt`.

Estos son los hechos internos que lo disparan:

updateType	Se publica cuando	details
STARTED	urbanServiceStarted , o se asigna inspector	vacío
PROGRESS	urbanServiceScheduled , urbanServiceDelayed , zoneNotServiced , treePruningScheduled , o cierra la inspección con acta	🔴 sin campo definido en la v1.5 para la fecha agendada — ver punto bloqueante
RESOLVED	urbanServiceCompleted , o cierra la inspección sin irregularidad	resolution.type + publicMessage
RETURNED	El reclamo no es de nuestra área. Vuelve a M2 para que lo re-derive	returnInfo.reasonCode
REJECTED	El expediente pasa a DISMISSED	cancellation.reasonCode

updateType	Se publica cuando	details
INFORMATION_REQUIRED	El inspector necesita un dato del vecino	informationRequest.messageForCitizen (+ requiredBy opcional). La v1.5 no usa informationRequestId

Publicamos ocho eventos y los ocho tienen consumidor declarado. Eran quince: los siete que le ofrecíamos a M2 como detalle del avance — `urbanServiceStarted`, `urbanServiceDelayed`, `urbanServiceCompleted`, `zoneNotServiced`, `containerOverflowed`, `environmentalInspectionScheduled` y `environmentalInspectionCompleted` — se quedaron sin consumidor cuando M2 definió que todo lo suyo entra por `updateTicketStatus`, y los sacamos del contrato. Los hechos siguen existiendo en nuestro modelo y son los que disparan cada `updateTicketStatus`, pero no salen al bus: un evento que nadie escucha es código que hay que escribir, versionar y mantener para nadie. Si alguno hace falta, se pide y se publica.

5.2 Los siete que no publicamos

El diseño queda escrito. Son los hechos que disparan cada `updateTicketStatus` y, si algún módulo los pide, se publican sin rediseñar nada.

`urbanServiceStarted`

```
serviceId, serviceTypeCode, mode,
startedAt, crewId, zoneIds[]
```

`urbanServiceDelayed`

```
serviceId, delayType, delayMinutes,
reason, newEstimatedEnd, detectedAt
```

Se publica una sola vez por servicio.

`urbanServiceCompleted`

```
serviceId, serviceTypeCode, mode, outcome,
startedAt, completedAt,
zoneResults[] { zoneId, status, reason? },
collection? { wasteType, volumeM3, weightKg, disposalSiteId },
attachments[], notes?, ticketId?
```

`zoneResults[]` solo en `mode = ROUTE`; `collection` solo en los servicios de recolección.

`zoneNotServiced`

```
serviceId, zoneId, neighborhoodIds[],
reason, notes?, proposedDate?, recordedAt
```

`neighborhoodIds[]` va explícito para que M2 ubique a los vecinos afectados sin conocer nuestras zonas.

`containerOverflowed`

containerId, containerCode, containerType,
zoneId, location,
detectedAt, ticketId?

environmentalInspectionScheduled → M2

inspectionId, serviceId, reportId, ticketId?,
reportType, zoneId, location,
scheduledDate, timeWindow { from, to }

environmentalInspectionCompleted → M2

inspectionId, reportId, ticketId?,
inspectedAt, outcome, publicSummary, nextStep

Los dos llevan la **proyección pública**: sin identidad del inspector, checklist, hallazgos internos ni contenido del acta. `publicSummary` es el texto que lee el vecino, y `nextStep` dice qué sigue —acta, reinspección o cierre— sin adelantar la sanción.

5.3 Enumeraciones

Diseño tentativo.

Enumeración	Valores
ServiceCategory	WASTE_COLLECTION , STREET_CLEANING , CONTAINERS , TREES , GREEN_SPACES , ENVIRONMENTAL_CONTROL
ServiceMode	ROUTE , POINT
ServiceStatus	SCHEDULED , RESCHEDULED , IN_PROGRESS , SUSPENDED , COMPLETED , PARTIALLY_COMPLETED , CANCELLED
ServiceOrigin	PLANNED , TICKET , WEATHER_ALERT , INSPECTION , MANUAL
TicketStatusUpdate	IN_PROGRESS , UPDATED , COMPLETED , REJECTED
DelayType	START , DURATION
ZoneResultStatus	SERVICED , PARTIAL , NOT_SERVICED
NotServicedReason	VEHICLE_BREAKDOWN , CREW_UNAVAILABLE , BLOCKED_ACCESS , STREET_CLOSURE , WEATHER , EXCESS_VOLUME , SECURITY_INCIDENT , OTHER
WasteType	HOUSEHOLD , RECYCLABLE , BULKY , GREEN , MIXED
DisposalSiteType	LANDFILL , TRANSFER_STATION , RECYCLING_PLANT , COMPOSTING_PLANT
ContainerType	HOUSEHOLD , RECYCLABLE , BULKY , GREEN
ContainerStatus	ACTIVE , OVERFLOWED , DAMAGED , UNDER_REPAIR , RELOCATING , REMOVED
DamageType	STRUCTURAL , BURNT , LID_BROKEN , WHEELS_BROKEN , VANDALIZED , MISSING

Enumeración	Valores
Severity	LOW , MEDIUM , HIGH , CRITICAL
TreeHealthStatus	HEALTHY , WEAKENED , DISEASED , DEAD
RiskLevel	NONE , LOW , MEDIUM , HIGH , CRITICAL
RiskType	FALLING_BRANCH , TRUNK_INSTABILITY , ROOT_UPLIFT , POWER_LINE_CONTACT , SIGN_OBSTRUCTION , PEST_INFESTATION
TreeInterventionType	FORMATION_PRUNING , SAFETY_PRUNING , REMOVAL , PLANTING , TREATMENT
TreeInterventionStatus	REQUESTED , PENDING_AUTHORIZATION , AUTHORIZED , REJECTED
GreenSpaceType	SQUARE , PARK , PLANTER , MEDIAN , PROMENADE
EnvironmentalReportType	NOISE , DUMPING , ILLEGAL_DUMPSITE , WATER_DISCHARGE , AIR_EMISSION , ODOR , PEST_INFESTATION , OTHER
EnvironmentalReportStatus	RECEIVED , UNDER_REVIEW , FORWARDED , DISMISSED , INSPECTION_SCHEDULED , INSPECTED , NO_VIOLATION , VIOLATION_FOUND , NOTICE_ISSUED , SANCTIONED , CLOSED
InspectionOutcome	NO_VIOLATION , VIOLATION_FOUND , INCONCLUSIVE
InspectionNextStep	NOTICE_TO_BE_ISSUED , REINSPECTION , CASE_CLOSED
ViolationType	NOISE_LIMIT , ILLEGAL_DUMPING , UNTREATED_DISCHARGE , HAZARDOUS_WASTE , AIR_EMISSION , NO_WASTE_MANAGEMENT , INSPECTION_OBSTRUCTION
SuggestedAction	WARNING , FORMAL_NOTICE , FINE , CLOSURE
SanctionDecision	FINE_ISSUED , CLOSURE_ORDERED , FORMAL_NOTICE_ISSUED , DISMISSED
CrewType	MUNICIPAL , COOPERATIVE , CONTRACTOR
VehicleType	COMPACTOR_TRUCK , DUMP_TRUCK , SWEEPER , WATER_TANKER , CRANE_TRUCK , VAN
Shift	MORNING , AFTERNOON , NIGHT
RepairDamageType	BROKEN_PAVEMENT , BROKEN_SIDEWALK , BROKEN_STREETLIGHT , BLOCKED_DRAIN , DAMAGED_STRUCTURE
StreetClosureType	TOTAL , PARTIAL

6. Eventos y datos que necesitamos recibir

Evento	Origen	Uso
<code>ticketUpdated</code>	M2	Según el <code>updateType</code> : <code>ROUTED</code> abre el expediente o el servicio, y el resto lo hace avanzar
<code>workOrderScheduled</code>	M3	Pasamos la solicitud de reparación a <code>IN_PROGRESS</code>
<code>workOrderCompleted</code>	M3	Cerramos la solicitud de reparación asociada
<code>commercialFineGenerated</code>	M4	Registramos la resolución del acta, pasamos a <code>SANCTIONED</code> y cerramos
<code>closureUpdate</code> (<code>status: ORDERED</code>)	M4	Ídem, con la clausura como resolución
<code>closureUpdate</code> (<code>status: LIFTED</code>)	M4	Registramos el levantamiento de la clausura y cerramos el expediente
<code>streetClosureApproved</code>	M7	Habilitamos la ejecución del servicio bloqueado
<code>streetClosureRejected</code>	M7	Reprogramamos o cancelamos el servicio dependiente
<code>streetClosureEnded</code>	M7	Liberamos la dependencia y habilitamos la reprogramación
<code>weatherAlertIssued</code>	Simulada por nosotros — nadie la publica hoy	Marcamos las zonas afectadas y disparamos la reprogramación masiva
<code>notificationSent</code>	M9	Registramos el acuse

6.1 Campos imprescindibles

Solo lo que necesitamos sí o sí. Cualquier campo adicional que ya publiquen lo aprovechamos, pero sin estos el flujo no funciona.

Origen	Evento o consulta	Campos imprescindibles
M2	<code>ticketUpdated</code> / <code>ROUTED</code>	<code>ticketId</code> , <code>citizenId</code> , <code>isAnonymous</code> , <code>responsibleAreaId</code>  , <code>requestType</code> , <code>summary</code> , <code>description</code> , <code>location</code> { <code>street</code> , <code>streetNumber</code> , <code>neighborhoodId</code> }  (confirmados en la v1.5)
M1	Consulta de ciudadano (<i>REST</i>)	<code>citizenId</code> , <code>fullName</code> , <code>documentId</code>
M1	Consulta de organización (<i>REST</i>)	<code>organizationId</code> , <code>legalName</code>
M3	<code>workOrderScheduled</code>	<code>sourceRequestId</code> 
M3	<code>workOrderCompleted</code>	<code>sourceRequestId</code>  , <code>outcome</code> , <code>completedAt</code>
M4	<code>commercialFineGenerated</code> , <code>closureUpdate</code> (<code>status: ORDERED</code> <code>LIFTED</code>)	<code>sourceViolationId</code>  , <code>decision</code> , <code>decidedAt</code>  (confirmado 24/08)
M7	<code>streetClosureApproved</code>	<code>streetClosureId</code> , <code>closureRequestId</code>  , <code>requestingModule</code>  , <code>startDate</code> , <code>endDate</code>

Origen	Evento o consulta	Campos imprescindibles
M7	<code>streetClosureRejected</code>	<code>closureRequestId</code> ✓, <code>rejectionReason</code> , <code>requestingModule</code> ✓
M7	<code>streetClosureEnded</code>	<code>streetClosureId</code> , <code>completionDateTime</code> — sin <code>closureRequestId</code>
M9	<code>notificationSent</code>	Referencia al evento que la originó

✓ **Los dos de M2 se cerraron con la v1.5.** `responsibleAreaId` ya viaja como campo común de `ticketUpdated`, y `location` ya viene estructurada con `neighborhoodId`.

✓ **Los tres de M7 se confirmaron el 25/08**, con nombres reales distintos a los tentativos que teníamos anotados (`closureRequestId` / `requestingModule`, no `sourceRequestId` / `sourceModule`). `streetClosureEnded` no trae el origen: correlaciona solo por `streetClosureId`, que hay que persistir desde el `streetClosureApproved`.

✓ **El tercero se cerró.** `sourceViolationId` en la resolución de M4 —el `violationId` que mandamos en el acta, sin el cual no sabíamos cuál resolvían y el expediente quedaba en `NOTICE_ISSUED` para siempre— ya viaja en `commercialFineGenerated` y `closureUpdate`, confirmado en el documento de referencia vigente de M4.

● **Uno nuevo apareció con la v1.5.** El campo común `progress` de `updateTicketStatus` es un `Int` (porcentaje), no la fecha agendada que necesitamos mandar, y no hay estructura de `details` definida para `STARTED` / `PROGRESS`.

7. Puntos bloqueantes

Punto	Con quién	Nuestra propuesta
Quién emite la identidad. El enunciado se la asigna a M9 y también a M1	M9, M1	M9 es el único emisor de JWT; M1 le informa los datos de acceso al registrar. El claim set se acuerda en la primera reunión: cada módulo emite con ese claim set en la primera entrega y migra al emisor del Core en la segunda
✅ <code>ticketUpdated / ROUTED</code> ya dice a qué módulo va y trae <code>location</code> estructurada	M2	Resuelto por la v1.5: <code>responsibleAreaId</code> , <code>citizenId</code> e <code>isAnonymous</code> son campos comunes, y <code>location</code> trae <code>neighborhoodId</code>
✅ M2 consume <code>updateTicketStatus</code>	M2	Confirmado en su contrato, ahora v1.5: evento único con <code>updateType</code> , y es lo único que consumen. Adoptamos su payload tal cual. Los siete eventos de dominio que dependían de M2 salieron del contrato publicado
<code>progress</code> de <code>updateTicketStatus</code> no sirve para nuestra fecha agendada	M2	Es un <code>Int</code> (porcentaje) y no hay <code>details</code> definido para <code>STARTED / PROGRESS</code> en la v1.5. Necesitamos saber cómo mandar la fecha/franja agendada del servicio
✅ M4 ya devuelve el identificador del acta	M4	<code>sourceViolationId</code> viaja en <code>commercialFineGenerated</code> y <code>closureUpdate</code> (antes <code>closureOrdered / closureLifted</code> , fusionados en un solo evento con <code>status: ORDERED LIFTED</code>). El caso "M4 decide no accionar" sigue sin disparar ningún evento; lo cerramos por vencimiento de un plazo configurable, dejándolo asentado
Zona contra barrio. El enunciado le da a M9 "barrios, zonas y catálogos globales"; nosotros necesitamos zonas operativas propias que agrupan barrios. La palabra es la misma y la cosa no	M9	M9 responsable del barrio, M6 de la zona operativa, que referencia <code>neighborhoodId[]</code> . Si M9 quiere quedarse con la palabra "zona", renombramos la nuestra